테스트 구조 및 방법론 분석

개요

Oasis CLI는 Go 표준 테스트 프레임워크를 기반으로 한 체계적인 테스트 구조를 가지고 있습니다. 이 문서는 프로젝트의 테스트 아키텍처, 테스트 전략, 사용되는 도구들, 그리고 모킹 전략에 대해 상세히 분석합니다.

테스트 아키텍처

전체 테스트 구조

```
oasis-cli/
├── cmd/
| ├── update_test.go # 업데이트 기능 통합 테스트
| └── common/
| └── wallet_test.go # 공통 지갑 유틸리티 테스트
├── wallet/
| ├── file/
| | ├── ed25519_test.go # Ed25519 암호화 단위 테스트
| | ├── sr25519_test.go # Sr25519 암호화 단위 테스트
| | └── secp256k1_test.go # Secp256k1 암호화 단위 테스트
| └── test/
| └── test.go # 테스트 계정 팩토리 및 유틸리티
```

```
└── build/
├── rofl/
| └── manifest_test.go # ROFL 매니페스트 검증 테스트
├── measurement/
| └── tdx_qemu_test.go # TDX QEMU 측정 테스트
└── dotenv/
└── dotenv_test.go # 환경 변수 처리 테스트
```

테스트 분류

1. 단위 테스트 (Unit Tests)

- 개별 함수나 메서드의 기능을 검증
- 외부 의존성을 최소화하여 격리된 환경에서 실행
- 주로 암호화 함수, 유틸리티 함수, 데이터 변환 로직 테스트

2. 통합 테스트 (Integration Tests)

- 여러 컴포넌트 간의 상호작용을 검증
- HTTP 서버 모킹, 파일 시스템 조작 등을 포함
- 실제 사용 시나리오에 가까운 테스트

3. 검증 테스트 (Validation Tests)

- 설정 파일, 매니페스트 파일 등의 유효성 검사
- 데이터 구조의 직렬화/역직렬화 검증

테스트 도구 및 프레임워크

핵심 테스트 라이브러리

1. Go 표준 테스트 프레임워크

```go
import "testing"

func TestFunction(t *testing.T) {
// 테스트 로직
}
```

2. Testify 라이브러리

```go
import "github.com/stretchr/testify/require"

func TestWithTestify(t *testing.T) {
require := require.New(t)
require.NoError(err)
require.Equal(expected, actual)
}
```

주요 Testify 기능 활용:

- `require.NoError()` : 에러 검증
- `require.Equal()` : 값 비교
- `require.EqualValues()` : 타입 변환 후 값 비교
- `require.ErrorContains()` : 에러 메시지 포함 여부 검증

- `require.Len()` : 슬라이스/맵 길이 검증
- `require.Contains()` : 컨테이너 포함 여부 검증

테스트 실행 환경

Makefile 기반 테스트 실행:

```
test-unit:
@$(PRINT) "$(CYAN)*** Running unit tests...$(OFF)\n"
@$(GO) test -v -race ./...

test: $(test-targets)
```

CI/CD 파이프라인:

- GitHub Actions를 통한 자동화된 테스트 실행
- 다중 플랫폼 지원 (Ubuntu, macOS, Windows)
- Go 1.24.x 버전 사용
- 의존성 캐싱을 통한 빌드 시간 최적화

테스트 전략 및 패턴

1. 테이블 기반 테스트 (Table-Driven Tests)

암호화 함수 테스트 예시:

```
func TestEd25519FromMnemonic(t *testing.T) {
mnemonics := []struct {
```

```
mnemonic  string

num  uint32

pubkey  string

valid  bool

}{

{

mnemonic: "equip will roof matter pink blind book anxiety banner elbow sun young

num: 0,

pubkey: "RWAfdhrxfbpQJDUp5ilzLxxY0I/92qhJEjhUBHVynYU=",

valid: true,

},

// ... 더 많은 테스트 케이스

}



for  _, m := range  mnemonics {

if  m.valid {

signer, _, err := Ed25519FromMnemonic(m.mnemonic, m.num)

require.NoError(t, err)

require.Equal(t, m.pubkey, signer.Public().String())

} else {

_, _, err := Ed25519FromMnemonic(m.mnemonic, 0)

require.Error(t, err)

}
```

```
}

}
```

장점:

- 다양한 입력 케이스를 체계적으로 테스트
- 테스트 케이스 추가가 용이
- 코드 중복 최소화

2. 주소 해석 테스트

복잡한 주소 형식 검증:

```
func TestResolveAddress(t *testing.T) {

for _, tc := range []struct {

address string

expectedAddr string

expectedEthAddr string

}{

{"oasis1qqzh32kr72v7x55cjnjp2me0pdn579u6as38kacz", "oasis1qqzh32kr72v7x55cjnjp2m

{"0x60a6321eA71d37102Dbf923AAe2E08d005C4e403", "oasis1qpaqumrpewltmh9mr73hteycfz

{"test:alice", "oasis1qrec770vrek0a9a5lcrv0zvt22504k68svq7kzve", ""},

// ... 더 많은 케이스

} {

addr, ethAddr, err := ResolveAddress(&net, tc.address)

if len(tc.expectedAddr) > 0 {
```

```
require.NoError(err, tc.address)

require.EqualValues(tc.expectedAddr, addr.String(), tc.address)

} else {

require.Error(err, tc.address)

}

}

}
```

3. 매니페스트 검증 테스트

단계별 검증 전략:

```
func  TestManifestValidation(t *testing.T) {

require := require.New(t)


// 빈 매니페스트는 유효하지 않음

m := Manifest{}

err := m.Validate()

require.ErrorContains(err, "name cannot be empty")


// 단계별로 필드를 채워가며 검증

m.Name = "my-simple-app"

err = m.Validate()

require.ErrorContains(err, "version cannot be empty")
```

```
// 최종적으로 모든 필드가 유효한 상태 검증
m.Deployments["default"].ParaTime = "bar"
err = m.Validate()
require.NoError(err)
}
```

모킹 전략

1. HTTP 서버 모킹

업데이트 기능 테스트에서의 HTTP 모킹:

```
func TestUpdate_PositiveFlow(t *testing.T) {
var ts *httptest.Server
ts = httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Req
switch r.URL.Path {
case "/repos/oasisprotocol/cli/releases/latest":
fmt.Fprint(w, makeReleaseJSON("v99.0.0", "body", "oasis-"+runtime.GOOS+"-"+runti
case "/asset":
// 실제 파일 서빙 로직
http.ServeFile(w, r, asset)
case "/asset.sha256":
// SHA256 체크섬 제공
```

```
fmt.Fprintln(w, sum)
default:
w.WriteHeader(http.StatusNotFound)
}
}))
defer ts.Close()

// 전역 변수 모킹
oldAPI := githubAPIBase
githubAPIBase = ts.URL
defer func() { githubAPIBase = oldAPI }()
}
```

2. 시스템 함수 모킹

실행 파일 경로 모킹:

```
func runTestUpdate(oasisBinaryPath string) int {
// os.Executable 함수 모킹
oldFn := osExecutable
osExecutable = func() (string, error) { return oasisBinaryPath, nil }
defer func() { osExecutable = oldFn }()
```

```
// 테스트 실행
rootCmd.ResetFlags()
rootCmd.SetArgs([]string{"update", "--yes"})
err := rootCmd.Execute()
if err != nil {
return 1
}
return 0
}
```

3. 파일 시스템 모킹

임시 디렉토리 활용:

```
func TestLoadManifest(t *testing.T) {
tmpDir, err := os.MkdirTemp("", "oasis-test-load-manifest")
require.NoError(err)
defer os.RemoveAll(tmpDir)

err = os.Chdir(tmpDir)
require.NoError(err)

// 테스트 파일 생성 및 검증
manifestFn := filepath.Join(tmpDir, "rofl.yml")
```

```
err = os.WriteFile(manifestFn, []byte(serializedYamlManifest), 0o600)

require.NoError(err)

}
```

테스트 유틸리티 및 헬퍼

1. 테스트 계정 팩토리

테스트 전용 계정 시스템:

```
type testAccountFactory struct{}


func (af *testAccountFactory) Kind() string {

return  Kind

}


func  NewTestAccount(testKey testing.TestKey) (wallet.Account, error) {

return &testAccount{testKey: testKey}, nil

}
```

특징:

- 실제 지갑 시스템과 동일한 인터페이스 구현
- 테스트 전용 키 생성 및 관리
- 다양한 암호화 알고리즘 지원

2. 테스트 데이터 생성

동적 테스트 데이터 생성:

```
func makeReleaseJSON(tag, body, assetName, assetURL string) string {
return fmt.Sprintf(`{
"tag_name": %q,
"body": %q,
"assets": [
{"name": %q, "browser_download_url": %q},
{"name": %q, "browser_download_url": %q}
]
}`, tag, body, assetName, assetURL, assetName+".sha256", assetURL+".sha256")
}
```

3. 파일 압축 유틸리티

테스트용 tar.gz 파일 생성:

```
func tarGz(t *testing.T, path, exeName string, content []byte) {
f, err := os.Create(path)
if err != nil {
t.Fatalf("create tarGz: %v", err)
}
gzw := gzip.NewWriter(f)
```

```
tw := tar.NewWriter(gzw)


hdr := &tar.Header{Name: exeName, Mode: 0o755, Size: int64(len(content))}
if err := tw.WriteHeader(hdr); err != nil {
t.Fatalf("hdr: %v", err)
}
// ... 압축 로직
}
```

테스트 커버리지 및 품질 관리

1. 테스트 실행 옵션

Race Condition 검출:

```
go test -v -race ./...
```

상세 출력 모드:

```
go test -v ./...
```

2. CI/CD 통합

GitHub Actions 워크플로우:

```
- name: Run tests
```

```
run: |
make test
- name: Check examples
if: runner.os != 'Windows'
run: |
make clean-examples
make examples
git diff --exit-code examples
```

다중 플랫폼 테스트:

- Ubuntu Latest
- macOS Latest
- Windows Latest

3. 테스트 격리

각 테스트의 독립성 보장:

- 임시 디렉토리 사용
- 전역 변수 백업 및 복원
- 테스트 후 정리 작업 (defer 활용)

테스트 방법론 분석

1. 긍정적 테스트 케이스

정상 동작 검증:

```
func TestUpdate_PositiveFlow(t *testing.T) {
// 정상적인 업데이트 프로세스 검증
exit := runTestUpdate(binPath)
if exit != 0 {
t.Fatalf("unexpected exit code %d", exit)
}
}
```

2. 부정적 테스트 케이스

에러 상황 검증:

```
func TestUpdate_AlreadyUpToDate(t *testing.T) {
// 이미 최신 버전인 경우의 동작 검증
if code := runTestUpdate(exe); code != 0 {
t.Fatalf("expected 0 exit, got %d", code)
}
}
```

3. 경계값 테스트

유효하지 않은 입력 검증:

```
{
key: "0x1f1455c61485737accdd610f5ea9ac1e4272c29b4c6c3189a349acc5bb598e7",
```

```
valid: false
}, // 너무 짧은 키
{
key: "0x1f1455c61485737accdd610f5ea9ac1e4272c29b4c6c3189a349acc5bb598e7d1",
valid: false
}, // 너무 긴 키
```

테스트 성능 및 최적화

1. 테스트 실행 시간 최적화

병렬 테스트 실행:

- Go의 기본 병렬 테스트 지원 활용
- CPU 집약적 테스트와 I/O 집약적 테스트 분리

의존성 캐싱:

- GitHub Actions에서 Go 모듈 캐싱
- 빌드 시간 단축

2. 리소스 관리

메모리 사용 최적화:

- 대용량 테스트 데이터의 효율적 관리
- 테스트 후 리소스 정리

파일 시스템 정리:

- 임시 파일 자동 삭제

- 테스트 격리를 위한 독립적인 작업 디렉토리

테스트 문서화 및 유지보수

1. 테스트 케이스 문서화

명확한 테스트 이름:

```
func TestEd25519FromMnemonic(t *testing.T) // 기능 중심 명명
func TestUpdate_PositiveFlow(t *testing.T) // 시나리오 중심 명명
func TestManifestValidation(t *testing.T) // 검증 중심 명명
```

2. 테스트 유지보수성

모듈화된 테스트 구조:

- 각 패키지별 독립적인 테스트
- 공통 테스트 유틸리티 분리
- 테스트 데이터와 로직 분리

버전 호환성:

- Go 버전 업그레이드 시 테스트 호환성 확인
- 의존성 라이브러리 업데이트 시 테스트 검증

결론

Oasis CLI의 테스트 구조는 다음과 같은 특징을 가집니다:

강점:

1. **체계적인 테스트 분류**: 단위, 통합, 검증 테스트의 명확한 구분
2. **효과적인 모킹 전략**: HTTP 서버, 파일 시스템, 시스템 함수 모킹
3. **테이블 기반 테스트**: 다양한 입력 케이스의 체계적 검증
4. **CI/CD 통합**: 자동화된 다중 플랫폼 테스트
5. **테스트 유틸리티**: 재사용 가능한 테스트 헬퍼 함수들

개선 가능한 영역:

1. **테스트 커버리지 측정**: 코드 커버리지 리포팅 도구 도입
2. **성능 테스트**: 벤치마크 테스트 추가
3. **E2E 테스트**: 전체 워크플로우 테스트 확장
4. **테스트 문서화**: 테스트 시나리오 문서화 강화

이러한 테스트 구조는 코드 품질 보장과 안정적인 기능 개발을 위한 견고한 기반을 제공합니다.